Computer Networks

CSA0712

CO5_Assessment Tool_1

Constrain Based Problem Solving

Q2. HTTP – Designing a Web Access Solution

Scenario

The company has an internal web application **intranet.local** that provides static content such as HTML, CSS, JavaScript and images, along with dynamic content generated by a backend server.

a) HTTP headers for caching, no-cache dynamic content, and minimizing round trips

1. Caching static files for 24 hours

For static files such as **HTML, CSS, JavaScript and images**, the server can send:

Cache-Control: public, max-age=86400

Here:

- public allows the response to be cached.
- max-age=86400 means the browser can cache the resource for **86,400 seconds = 24 hours**.

Example:

HTTP/1.1 200 OK

Cache-Control: public, max-age=86400

The server can also include:

Expires: <date/time 24 hours in the future>

However, Cache-Control is the preferred modern mechanism.

2. Preventing caching of dynamic content

For dynamic pages/data, the server should send:

Cache-Control: no-store

This tells the browser **not to store the response in its cache**.

Example:

HTTP/1.1 200 OK

Cache-Control: no-store

For stricter compatibility, the server can also use:

Pragma: no-cache

3. Server identification header

Every response must contain:

Server: IntraWeb/1.0

This identifies the web server as required by the scenario.

4. Minimizing round trips

To reduce unnecessary round trips, the application should use HTTP persistent connections (keep-alive). Multiple resources can be requested over the same TCP connection instead of creating a new connection for every file.

For HTTP/2 over HTTPS, multiplexing can further reduce the number of round trips by allowing multiple resources to be transferred concurrently over one connection.

b) Sequence of HTTP requests and responses

The user accesses:

http://intranet.local/dashboard

for the **first time**.

The page contains:

- 1 HTML file
- 1 CSS file
- 1 JavaScript file
- 1 image file

Step 1: DNS resolution

The browser first resolves:

intranet.local → internal IP address

The DNS server is an **internal company DNS server**, so the application is accessible only inside the company network.

For example:

intranet.local → 192.168.10.20

The exact IP address depends on the company's network configuration.

Step 2: Establish TCP connection

The browser establishes a TCP connection with the web server on the default HTTP port:

Client → Server

TCP connection → Port 80

Using a persistent connection allows multiple HTTP requests to use the same connection.

Step 3: Request HTML page

The browser sends:

GET /dashboard HTTP/1.1

Host: intranet.local

Connection: keep-alive

The server returns:

HTTP/1.1 200 OK

Server: IntraWeb/1.0

Content-Type: text/html

Cache-Control: no-store

Connection: keep-alive

The HTML page is dynamic, so it is **not cached**.

Step 4: Browser discovers required resources

After receiving the HTML, the browser identifies the required resources:

/dashboard

├── style.css

├── script.js

└── image.jpg

The browser requests these resources using the **same persistent connection**.

For example:

GET /style.css HTTP/1.1

Host: intranet.local

Response:

HTTP/1.1 200 OK

Server: IntraWeb/1.0

Content-Type: text/css

Cache-Control: public, max-age=86400

Step 5: Request JavaScript

GET /script.js HTTP/1.1

Host: intranet.local

Response:

HTTP/1.1 200 OK

Server: IntraWeb/1.0

Content-Type: application/javascript

Cache-Control: public, max-age=86400

Step 6: Request image

GET /image.jpg HTTP/1.1

Host: intranet.local

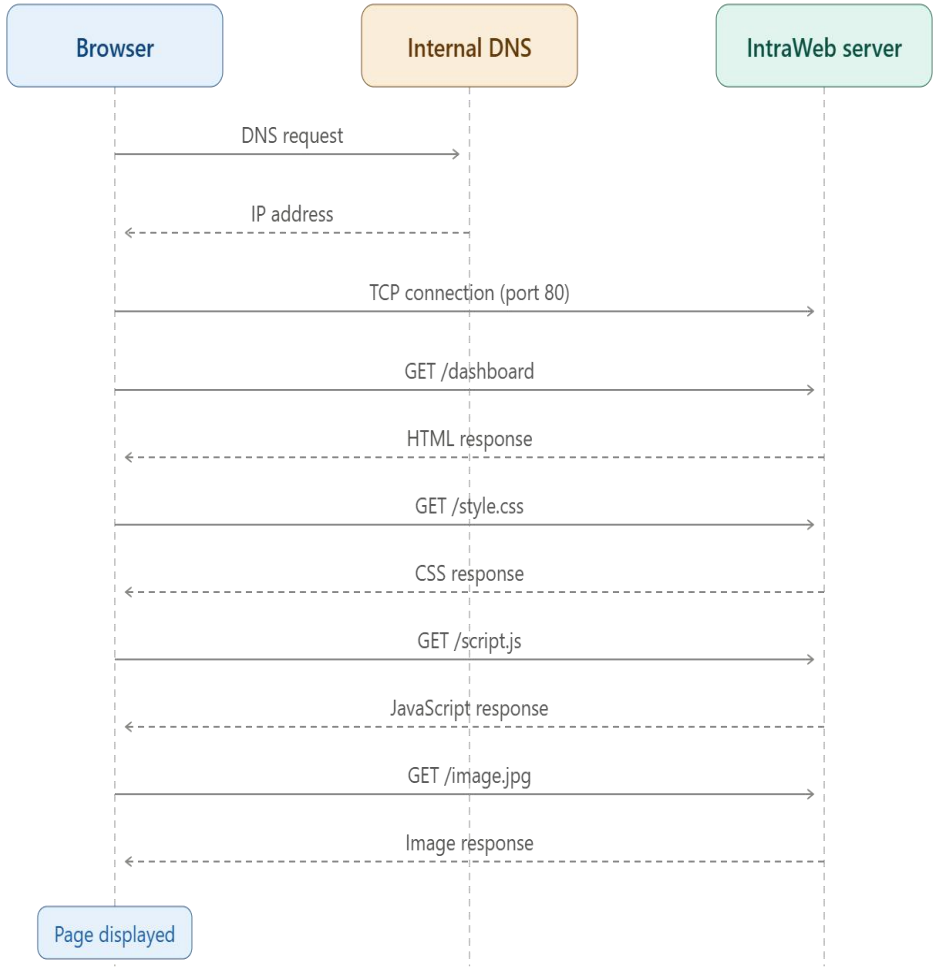
Response:

HTTP/1.1 200 OK

Server: IntraWeb/1.0

Content-Type: image/jpeg

Cache-Control: public, max-age=86400



How the design minimizes round trips

The design minimizes round trips by:

1. Using **one persistent TCP connection**.
2. Reusing the connection for multiple HTTP requests.
3. Avoiding separate TCP connections for each resource.

4. Caching static resources for **24 hours**, so subsequent page loads do not need to download them again.
5. If HTTP/2 is used with HTTPS later, multiplexing can allow the CSS, JavaScript and image requests to be transferred concurrently.

Therefore, the **first visit** downloads all required resources, while **later visits** can reuse the cached static resources.

c) Changes required when HTTPS is enabled

If the company later enables secure access through HTTPS, the application would need several additional changes.

1. Obtain and install a TLS certificate

The server needs a valid **TLS/SSL certificate** for:

intranet.local

For an internal application, the company can use an **internal Certificate Authority (CA)**, provided company devices trust that CA.

2. Configure HTTPS

The web server must be configured to support HTTPS using TLS.

HTTPS normally uses:

Port 443

instead of HTTP port 80.

3. Keep the same caching headers

The caching requirements do not change.

For static resources:

Cache-Control: public, max-age=86400

For dynamic content:

Cache-Control: no-store

Therefore, HTTPS does not change the caching policy.

4. Keep the Server header

Every HTTPS response should still contain:

Server: IntraWeb/1.0

This satisfies the server-identification requirement.

5. Internal-only access

The application should remain accessible only through the **company's internal network**.

This can be enforced using:

- Internal DNS
- Firewall rules
- Network access controls
- Internal routing

The HTTPS server should not be exposed publicly to the Internet.

6. Redirect HTTP to HTTPS

If HTTP is still enabled, requests to port 80 can be redirected to HTTPS:

HTTP/1.1 301 Moved Permanently

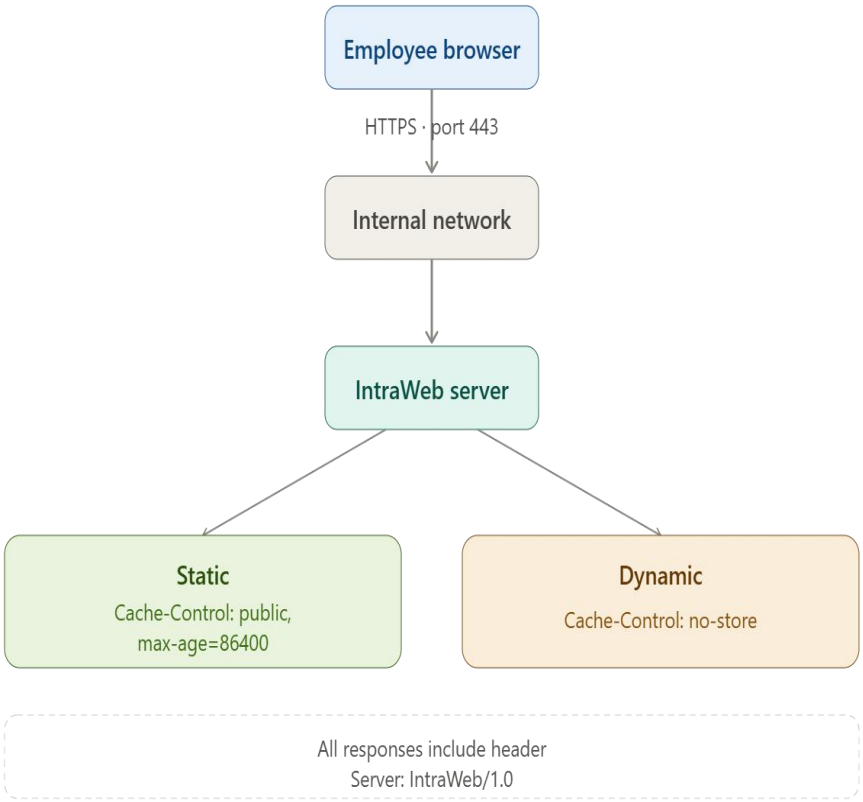
Location: https://intranet.local/dashboard

This ensures users use the secure connection.

7. Use persistent connections / HTTP/2

HTTPS can use a persistent connection, and the server can support **HTTP/2**. HTTP/2 provides multiplexing, allowing multiple resources such as CSS, JavaScript and images to be transferred efficiently over the same connection.

Final HTTPS design



Conclusion

The proposed HTTP solution keeps the application **internal-only**, caches static resources for **24 hours**, prevents caching of dynamic content, identifies the server using IntraWeb/1.0, and uses persistent connections to reduce unnecessary round trips. When HTTPS is introduced, TLS certificates, HTTPS configuration, port 443, and internal security controls must be added while retaining the same caching and server-header requirements.